

Capítulo 21

Programação Orientada a Objetos: Polimorfismo

Exercícios — Resolução Didática

“Resolvemos os 14 exercícios 21.3 a 21.16. Os primeiros oito são conceituais; o próprio livro deixa claro que os últimos seis (21.12 a 21.16) são aplicações consolidadoras das hierarquias dos capítulos anteriores — agora com *virtual*, *override* e *dynamic_cast*, tornando o código verdadeiramente polimórfico. Todos os códigos compilam com `g++ -Wall -Wextra -std=c++14` sem warnings.”

1 Exercício 21.3 — Programação geral vs específica

Enunciado. Como o polimorfismo permite que você trabalhe uma programação “geral” em vez de uma “específica”? Discuta as principais vantagens.

O contraste

Programação *específica* é a programação em que cada tipo de objeto exige código distinto:

```
// Programacao especifica: cada tipo, um caminho de codigo
for (Funcionario *f : folha) {
    if (typeid(*f) == typeid(FuncionarioAssalariado))
        pagamento = ((FuncionarioAssalariado*)f)->getSalario() * 4;
    else if (typeid(*f) == typeid(FuncionarioHorista))
        pagamento = calcularHorista((FuncionarioHorista*)f);
    // ... mais cinco else-ifs ...
}
```

Programação *geral* (*polimórfica*) trata todos os tipos uniformemente, com uma só linha:

```
// Programacao geral: uma so linha trata todos os tipos
for (Funcionario *f : folha) {
    pagamento = f->earnings();    // virtual dispatch
}
```

Três vantagens da programação geral

1. Código cliente não precisa conhecer a hierarquia

O loop processador da folha de pagamento (no nosso 21.12) *não menciona* `FuncionarioAssalariado`, `FuncionarioHorista`, etc. Trabalha exclusivamente através da interface `Funcionario`. Adicionar um novo tipo de funcionário (digamos, `FuncionarioPensionista`) não requer mudar uma *linha sequer* do loop.

2. Manutenção reduzida

Cada modificação na fórmula de pagamento de um tipo fica *em uma classe só*, a classe daquele tipo. Em programação específica, mudar a fórmula de funcionários horistas exigiria revisar todos os pontos onde aparecem `if (tipo == HORISTA)`.

3. Convida o código a ser correto por construção

Programação específica com `switch` sobre tipos exige *lembrar* de adicionar um `case` para cada novo tipo. Esquecer um é bug latente. Em programação geral, esquecer-se de implementar uma virtual pura é *erro de compilação*: a classe permaneceria abstrata.

Observação de Engenharia de Software

A programação polimórfica é uma manifestação do *Princípio Aberto-Fechado* (Open-Closed Principle): módulos devem ser *abertos para extensão* (acrescentar novos tipos) mas *fechados para modificação* (não tocar código cliente já testado). Polimorfismo viabiliza esse princípio: novos tipos são acomodados criando-se novas classes, sem alterar código cliente preexistente.

2 Exercício 21.4 — Problemas do `switch`

Enunciado. Discuta os problemas de programação com a lógica de `switch`. Explique por que o polimorfismo pode ser uma alternativa eficaz.

Quatro problemas com `switch` sobre tipos

1. Repetição

Cada `switch` sobre tipo enumerado precisa cobrir *todos* os casos. Se há 10 tipos e 5 operações diferentes que dependem do tipo, você terá 5 `switch` de 10 ramos cada. Acrescentar um 11º tipo significa atualizar todos os 5 `switch`.

2. Esquecimento

O compilador C++ por padrão *não* alerta quando um `switch` sobre um `enum` omite um valor. Você adiciona o 11º tipo, esquece de adicionar um `case` em um `switch`, e o código silenciosamente trata o novo tipo como se fosse o caso `default`. (`-Wswitch-enum` ajuda, mas não é padrão.)

3. Dispersão da lógica

A lógica específica de cada tipo fica *espalhada* por toda a base de código, em todos os `switch` que mencionam o tipo. Em vez de “o que faz o tipo X?” ser respôndível olhando uma classe, exige percorrer dezenas de funções.

4. Acoplamento ao conjunto fechado de tipos

O `switch` pressupõe que você *sabe quais são todos os tipos*. Bibliotecas que querem permitir que *clientes* adicionem novos tipos não podem usar `switch` — precisam abstrair.

O polimorfismo resolve os quatro

- **Repetição:** cada operação é uma função virtual; nenhuma duplicação.
- **Esquecimento:** funções virtuais puras forçam implementação; esquecer é erro de compilação.

- **Dispersão:** toda a lógica do tipo X está na classe X.
- **Acoplamento:** bibliotecas exportam apenas a classe-base; clientes derivam novos tipos.

Boa Prática de Programação

Heurística: se você se vê prestes a escrever um `switch` sobre um campo `tipo` de uma estrutura, pare. Provavelmente quer uma hierarquia de classes com funções virtuais. Exceções legítimas: `switch` sobre `enum` com casos pequenos e mutuamente exclusivos onde não há comportamento “geral” a ser fatorado.

3 Exercício 21.5 — Herança de interface vs implementação

Enunciado. Faça a distinção entre herdar interface e herdar implementação. Como as hierarquias diferem entre os dois casos?

Definições

Herança de interface: a classe-base define *somente o que* suas derivadas devem fazer — só virtuais puras, nenhuma implementação. Cada derivada implementa por si própria.

Herança de implementação: a classe-base oferece código concreto que as derivadas reutilizam. Pode haver virtuais que as derivadas refinam, mas há substância compartilhada.

Características de hierarquias de *interface*

- Classes-base são *puramente abstratas* (todas as funções = 0).
- Exemplo protótipo: `EmissaoCarbono` (Ex. 21.17) — nenhum dado, apenas o método `retEmissaoCarbono()` = 0.
- Em Java/C#, isso seria `interface`, não `class`.
- Derivadas não se beneficiam de nenhum código reaproveitado — apenas do contrato.

Características de hierarquias de *implementação*

- Classes-base têm código substancial (talvez funções virtuais com corpos, dados-membro, funções não-virtuais).
- Exemplo protótipo: `Pacote` → `PacoteDoisDias` (Ex. 21.15) — a derivada herda os dados de remetente/destinatário, getters, e a versão base de `calculaCusto()` que é refinada.
- Existe perigo de “software frágil” (Ex. 20.5): mudanças na base podem quebrar derivadas.

Exemplo comparativo da nossa coleção

Hierarquia	Tipo	Justificativa
<code>EmissaoCarbono</code> (21.17)	Interface pura	<code>Predio</code> , <code>Carro</code> , <code>Bicicleta</code> não compartilham dados, só o contrato <code>retEmissaoCarbono</code> .
<code>Forma</code> (21.13)	Híbrida	A camada raiz <code>Forma</code> só oferece o nome (mínimo). As intermediárias acrescentam contratos.
<code>Funcionario</code> (21.12)	Implementação parcial	Dados de identificação (nome, CPF) são herdados; só <code>earnings</code> é pura.
<code>Pacote</code> (21.15)	Implementação	A base oferece valores default; as derivadas refinam.

Observação de Engenharia de Software

A regra do dedo polegar: herança de interface escala melhor quando há muitas implementações muito diferentes (pense em `Iterator` em Java, sem código, dezenas de classes a implementam). Herança de implementação funciona melhor quando há um “template” de comportamento que algumas derivadas ajustam levemente. C++ permite os dois; Java tem ambos como construtos separados (`interface` e `abstract class`).

4 Exercício 21.6 — O que são funções virtuais?

Enunciado. O que são funções virtuais? Descreva uma circunstância em que seriam apropriadas.

Definição

Uma **função virtual** é uma função-membro declarada na classe-base com a palavra-chave `virtual`, indicando que *derivadas podem sobrescrevê-la* e que chamadas através de ponteiros/referências à base serão *despachadas dinamicamente* para a versão da derivada.

```
class Funcionario {
public:
    virtual double earnings() const = 0;    // pura (a base nao implementa)
    virtual void print() const;            // com implementacao default
    virtual ~Funcionario() {}              // destrutor virtual
};
```

Circunstância apropriada

Sempre que houver:

1. Uma *hierarquia* de classes;
2. Um *contrato* (uma função) que muitas das derivadas implementam de modos distintos;
3. Código cliente que vai chamar essa função através de *ponteiros ou referências à classe-base*.

Exemplos da nossa coleção

- `Funcionario::earnings()` (21.12) — cada tipo de funcionário calcula pagamento de modo distinto, mas o loop da folha de pagamento usa só `f.earnings()`.
- `Forma::print()` (21.13) — cada forma se apresenta de modo distinto, mas o gerenciador de tela chama só `f->print()`.
- `Pacote::calculaCusto()` (21.15) — cada tipo de pacote tem sua fórmula, mas o sistema de remessa chama só `p->calculaCusto()`.

5 Exercício 21.7 — Vinculação estática vs dinâmica; *vtable*

Enunciado. Faça a distinção entre vinculação estática e vinculação dinâmica. Explique o uso de funções virtuais e da *vtable* na vinculação dinâmica.

Vinculação estática vs dinâmica

Vinculação estática (*early binding*): o compilador decide qual função chamar *em tempo de compilação*, com base no tipo declarado da variável. É o que acontece com chamadas não-virtuais.

Vinculação dinâmica (*late binding*): o programa decide qual função chamar *em tempo de execução*, com base no tipo real do objeto. É o que acontece com chamadas virtuais via ponteiros/referências à base.

```
class Base {
public:
    void f() { cout << "Base::f"; }    // nao virtual
    virtual void g() { cout << "Base::g"; } // virtual
};
```

```
};
class Derivada : public Base {
public:
    void f() { cout << "Derivada::f"; }
    void g() override { cout << "Derivada::g"; }
};

Derivada d;
Base *p = &d;
p->f();    // Base::f (estatica - tipo declarado de p eh Base)
p->g();    // Derivada::g (dinamica - tipo real eh Derivada)
```

Como a vinculação dinâmica funciona internamente: a *vtable*

Cada classe com funções virtuais ganha, em tempo de compilação, uma **tabela virtual** (*vtable*): uma tabela estática de ponteiros para as implementações das funções virtuais daquela classe.

Cada *objeto* de uma classe com virtuais tem um ponteiro escondido (*vp**tr*) que aponta para a *vtable* da sua classe.

Quando o código executa `p->g()`:

1. Localiza o *vp**tr* dentro do objeto apontado por `p`.
2. Lê a *vtable* apontada.
3. Encontra a entrada da função `g` na *vtable*.
4. Salta para o código apontado por essa entrada.

Diagrama esquemático



Implicações práticas

- **Custo de memória:** cada objeto polimórfico aumenta de tamanho em um ponteiro (geralmente 8 bytes).
- **Custo de tempo:** cada chamada virtual exige uma indireção a mais (lookup na *vtable*). Em código crítico, isso pode importar.
- **Inicialização:** construtores e destrutores configuram o *vp**tr* corretamente para o tipo da classe atual — é por isso que polimorfismo *não funciona* dentro dos construtores.

A sobrecarga do polimorfismo (*vp**tr* + indireção) é geralmente *desprezível* na maioria das aplicações. O ganho em manutenibilidade compensa amplamente. Mas em loops apertados (jogos, simulações numéricas) onde cada nanossegundo conta, considere designs alternativos como *static polymorphism* via templates (CRTP), ou *type erasure* controlado.

6 Exercício 21.8 — Virtuais vs virtuais puras

Enunciado. Faça a distinção entre funções virtuais e funções virtuais puras.

Sintaxe e semântica

Função virtual:

```
virtual void f();           // tem corpo (no .cpp)
```

Função virtual pura:

```
virtual void f() = 0;       // SEM corpo na base
```

Implicações

Aspecto	Virtual	Virtual pura
Sintaxe na base	<code>virtual void f();</code>	<code>virtual void f() = 0;</code>
Implementação na base obrigatória?	SIM (corpo no .cpp)	NÃO (mas mesmo havendo, derivadas devem implementar)
Derivada precisa sobrescrever?	NÃO (herda da base)	SIM (senão a derivada também é abstrata)
Classe pode ser instanciada?	SIM	NÃO (abstrata)
Despacho dinâmico?	SIM	SIM

Por que ambas existem?

Virtual com implementação: útil quando há um comportamento *default* sensato que a maioria das derivadas pode herdar, e poucas precisam refinar.

Virtual pura: útil quando não há um default razoável; cada derivada *tem* que pensar na sua própria versão.

Exemplo

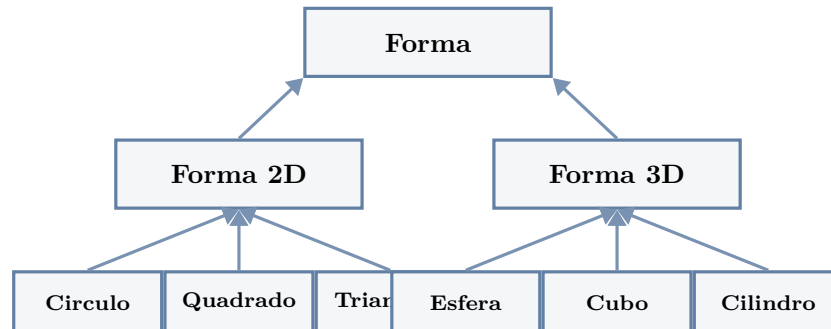
No nosso `Funcionario` (21.12), as duas convivem:

- `earnings()` é **pura**: não há default razoável; assalariados, horistas, comissionados têm formulas radicalmente diferentes.
- `print()` é **virtual (com impl.)**: a base sabe imprimir nome+CPF+nascimento; as derivadas refinam para adicionar campos específicos — mas se uma derivada esquecesse de sobrescrever, ainda imprimiria algo útil.

7 Exercício 21.9 — Classes-base abstratas na hierarquia Forma

Enunciado. Sugira um ou mais níveis de classes-base abstratas para a hierarquia Forma.

Sugestão



Níveis de abstração

Nível 1 — Forma: classe-base abstrata raiz. Funcionalidade comum a *todas* as formas (2D ou 3D): nome, talvez cor, posição no espaço. Contém *peelo menos* uma virtual pura, como `print()`.

Nível 2 — FormaBidimensional: classe-base abstrata para todas as formas planas. Acrescenta a virtual pura `retArea()`.

Nível 2 (lado direito) — FormaTridimensional: classe-base abstrata para todos os sólidos. Acrescenta `retArea()` (área de superfície) e `retVolume()`.

Nível 3 — classes concretas: Circulo, Quadrado, Triangulo, Esfera, Cubo, Cilindro.

Esboço em C++

```

class Forma {
public:
    virtual ~Forma() {}
    virtual std::string getNome() const = 0;
    virtual void print() const = 0;
};

class FormaBidimensional : public Forma {
public:
    virtual double retArea() const = 0;
};

class FormaTridimensional : public Forma {
public:
    virtual double retArea() const = 0;
    virtual double retVolume() const = 0;
};

class Circulo : public FormaBidimensional {
    // implementa retArea, print
};

class Esfera : public FormaTridimensional {
    // implementa retArea, retVolume, print
};
  
```

Implementaremos isso completamente no Exercício 21.13.

8 Exercício 21.10 — Polimorfismo e extensibilidade

Enunciado. Como o polimorfismo promove a extensibilidade?

Mecanismo da extensão

Para acrescentar um novo tipo a uma hierarquia polimórfica:

1. Defina uma nova classe que deriva da classe-base apropriada.
2. Implemente as funções virtuais puras.
3. *Pronto.* Nenhum código cliente existente precisa ser tocado.

Exemplo concreto

Considere nosso sistema de folha de pagamento (21.12). Imagine que o RH agora quer um novo tipo: `FuncionarioEstagiario`, com salário fixo, mas só se as horas trabalhadas excederem 30. Para acomodar:

```
class FuncionarioEstagiario : public Funcionario {
public:
    FuncionarioEstagiario(const std::string &n, const std::string &s,
                          const std::string &c, const Date &nasc,
                          double bolsa, double horasMensais)
        : Funcionario(n, s, c, nasc),
          bolsa(bolsa), horas(horasMensais) {}

    double earnings() const override {
        return horas >= 30 ? bolsa : bolsa * (horas / 30.0);
    }
    void print() const override {
        cout << "Estagiario: ";
        Funcionario::print();
        cout << "\n  Bolsa: R$ " << bolsa << ", horas: " << horas;
    }
private:
    double bolsa;
    double horas;
};
```

Acrescentamos esse arquivo e *nada mais muda*:

```
folha.push_back(new FuncionarioEstagiario("Bia", "Lima", "555.555.555-55",
    Date(2, 14, 2003), 1500, 25));
// O loop processador da folha de pagamento (intocado!) ja' funciona.
```

Compare com o cenário sem polimorfismo: teríamos que adicionar um `case ESTAGIARIO` em todos os `switch` do código cliente, e essa modificação em código testado é exatamente o que queremos evitar.

9 Exercício 21.11 — Polimorfismo num simulador de voo

Enunciado. Você foi escolhido para desenvolver um simulador de voo com saídas gráficas elaboradas. Explique por que a programação polimórfica poderia ser especialmente eficaz.

Tipos de objetos em um simulador de voo

Um simulador realista contém dezenas de tipos de objetos:

- *Aeronaves*: bois comerciais, helicópteros, planadores, jatos militares, drones.
- *Terreno*: polígonos de relevo, edificações, estradas, rios.
- *Vegetação*: árvores, arbustos, ervas.
- *Atmosfera*: nuvens, neblina, precipitação, ventos.
- *Outros tráfegos*: outros aviões simulados.

Cada um precisa, no mínimo, ser desenhado a cada *frame*, e atualizado a cada passo de tempo. Sem polimorfismo, o código central seria um `switch` gigantesco com centenas de casos.

Hierarquia polimórfica

```
class ObjetoSimulado {
public:
    virtual ~ObjetoSimulado() {}
    virtual void desenhar(Camera &cam) const = 0;
    virtual void atualizar(double dt) = 0;
    virtual BoundingBox limites() const = 0;
};

class Avi~ao      : public ObjetoSimulado { /* ... */ };
class Edif~icio  : public ObjetoSimulado { /* ... */ };
class Nuvem      : public ObjetoSimulado { /* ... */ };
```

Loop principal do simulador

```
while (rodando) {
    double dt = obterDeltaTempo();
    for (auto &obj : mundo) {           // mundo eh vector<unique_ptr<ObjetoSimulado>>
        obj->atualizar(dt);              // POLIMORFICO
    }
    limparTela();
    for (auto &obj : mundo) {
        obj->desenhar(camera);          // POLIMORFICO
    }
    apresentarFrame();
}
```

Quatro vantagens específicas para o simulador

1. **Extensibilidade**: adicionar balão de ar quente é uma nova classe; não requer mudar o motor.
2. **Modularidade**: cada tipo de objeto é seu próprio arquivo; modificações ficam contidas.
3. **Coleções uniformes**: um só `vector<ObjetoSimulado*>` contém todos os tipos.
4. **Código de espaço de desenho**: algoritmos como “ordenar objetos por distância à câmera para renderização transparente” funcionam sobre qualquer objeto polimórfico, sem conhecer o tipo concreto.

Observação de Engenharia de Software

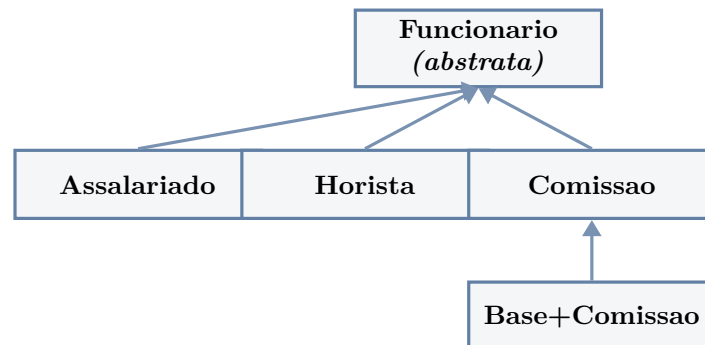
Engines de jogos profissionais (Unreal, Unity, Godot) usam polimorfismo extensivamente para exatamente este tipo de problema — combinado com *Entity-Component Systems* (ECS) quando a estrutura “é-um” não basta e “tem-uns” são mais expressivos.

10 Exercício 21.12 — Folha de pagamento com bonus de aniversário

Enunciado. Modifique o sistema de folha de pagamento (figs. 21.13–21.23) para incluir `dataNascimento` (do tipo `Date`) em `Funcionario`. Suponha que a folha seja processada uma vez por mês. Use `vector` de referências a `Funcionario`. No loop, adicione bônus de R\$ 100 se o mês atual for o de aniversário.

Análise do problema

A hierarquia polimórfica de funcionários é composta por:



A classe `Funcionario` agora tem `dataNascimento` (tipo `Date`). O método `earnings()` continua sendo virtual pura.

Classe `Date` (reaproveitada do Cap. 18)

```

1  // Date.h - reaproveitado do Cap. 18 (simplificado)
2  #ifndef DATE_H
3  #define DATE_H
4
5  #include <string>
6  #include <iostream>
7
8  class Date {
9      friend std::ostream &operator<<(std::ostream &out, const Date &d);
10 public:
11     Date(int mes = 1, int dia = 1, int ano = 2000);
12
13     int getMes() const;
14     int getDia() const;
15     int getAno() const;
16
17 private:
18     int mes, dia, ano;
19
20     static bool ehBissexto(int a);
21     static int diasDoMes(int m, int a);
22 };
23
24 #endif
  
```

Classe-base `Funcionario`

```

1  // Exercício 21.12 - Funcionario.h
2  // Classe-base ABSTRATA: tem funcao virtual pura earnings().
3  // Inclui dataNascimento (acrescimo do exercicio 21.12).
4  #ifndef FUNCIONARIO_H
5  #define FUNCIONARIO_H
6
7  #include <string>
8  #include <iostream>
9  #include "Date.h"
10
11 class Funcionario {
12 public:
13     Funcionario(const std::string &primeiroNome,
14                 const std::string &sobrenome,
15                 const std::string &cpf,
16                 const Date &dataNascimento);
17
18     virtual ~Funcionario() {}           // destrutor virtual (Sec. 21.9)
19
20     void setPrimeiroNome(const std::string &p);
21     std::string getPrimeiroNome() const;
22     void setSobrenome(const std::string &s);
23     std::string getSobrenome() const;
24     void setCPF(const std::string &c);
25     std::string getCPF() const;
26
27     const Date &getDataNascimento() const;
28
29     // == FUNCAO VIRTUAL PURA ==
30     // Cada derivada concreta deve implementar sua propria formula
31     virtual double earnings() const = 0;
32
33     // virtual nao-pura: derivadas podem sobrepor para refinar
34     virtual void print() const;
35
36 private:
37     std::string primeiroNome;
38     std::string sobrenome;
39     std::string cpf;
40     Date dataNascimento;
41 };
42
43 #endif

```

```

1  // Exercício 21.12 - Funcionario.cpp
2  #include "Funcionario.h"
3  using namespace std;
4
5  Funcionario::Funcionario(const string &p, const string &s,
6                           const string &c, const Date &nasc)
7      : primeiroNome(p), sobrenome(s), cpf(c), dataNascimento(nasc) {}
8
9  void Funcionario::setPrimeiroNome(const string &p) { primeiroNome = p; }
10 string Funcionario::getPrimeiroNome() const { return primeiroNome; }
11 void Funcionario::setSobrenome(const string &s) { sobrenome = s; }
12 string Funcionario::getSobrenome() const { return sobrenome; }
13 void Funcionario::setCPF(const string &c) { cpf = c; }
14 string Funcionario::getCPF() const { return cpf; }
15
16 const Date &Funcionario::getDataNascimento() const { return dataNascimento; }
17
18 void Funcionario::print() const {
19     cout << primeiroNome << " " << sobrenome

```

```

20         << "\n CPF: " << cpf
21         << "\n Nascimento: " << dataNascimento;
22     }

```

Quatro derivadas concretas

```

1  // Exercício 21.12 - FuncionarioAssalariado.h
2  #ifndef FUNCASSALARIADO_H
3  #define FUNCASSALARIADO_H
4
5  #include "Funcionario.h"
6
7  class FuncionarioAssalariado : public Funcionario {
8  public:
9      FuncionarioAssalariado(const std::string &p, const std::string &s,
10                          const std::string &c, const Date &nasc,
11                          double salarioSemanal);
12
13      void setSalarioSemanal(double s);
14      double getSalarioSemanal() const;
15
16      // Sobrepoem virtuais (override - palavra-chave do C++11)
17      double earnings() const override;
18      void print() const override;
19
20 private:
21     double salarioSemanal;
22 };
23
24 #endif

```

```

1  // Exercício 21.12 - FuncionarioAssalariado.cpp
2  #include <iostream>
3  #include <iomanip>
4  #include "FuncionarioAssalariado.h"
5  using namespace std;
6
7  FuncionarioAssalariado::FuncionarioAssalariado(
8      const string &p, const string &s, const string &c,
9      const Date &nasc, double salario)
10      : Funcionario(p, s, c, nasc) {
11      setSalarioSemanal(salario);
12  }
13
14  void FuncionarioAssalariado::setSalarioSemanal(double s) {
15      salarioSemanal = (s >= 0) ? s : 0;
16  }
17  double FuncionarioAssalariado::getSalarioSemanal() const {
18      return salarioSemanal;
19  }
20
21  // 4 semanas por mes (aproximacao didática)
22  double FuncionarioAssalariado::earnings() const {
23      return salarioSemanal * 4;
24  }
25
26  void FuncionarioAssalariado::print() const {
27      cout << "Assalariado: ";
28      Funcionario::print();
29      cout << "\n Salario semanal: R$ "
30          << fixed << setprecision(2) << salarioSemanal;
31  }

```

```

1 // Exercício 21.12 - FuncionarioHorista.h
2 #ifndef FUNCHORISTA_H
3 #define FUNCHORISTA_H
4
5 #include "Funcionario.h"
6
7 class FuncionarioHorista : public Funcionario {
8 public:
9     FuncionarioHorista(const std::string &p, const std::string &s,
10                        const std::string &c, const Date &nasc,
11                        double salarioPorHora, double horasSemanais);
12
13     void setSalarioPorHora(double s); // >= 0
14     double getSalarioPorHora() const;
15     void setHoras(double h); // 0 <= h <= 168 (horas/sem)
16     double getHoras() const;
17
18     double earnings() const override;
19     void print() const override;
20
21 private:
22     double salarioPorHora;
23     double horasSemanais;
24 };
25
26 #endif

```

```

1 // Exercício 21.12 - FuncionarioHorista.cpp
2 #include <iostream>
3 #include <iomanip>
4 #include "FuncionarioHorista.h"
5 using namespace std;
6
7 FuncionarioHorista::FuncionarioHorista(
8     const string &p, const string &s, const string &c,
9     const Date &nasc, double sh, double hs)
10 : Funcionario(p, s, c, nasc) {
11     setSalarioPorHora(sh);
12     setHoras(hs);
13 }
14
15 void FuncionarioHorista::setSalarioPorHora(double s) {
16     salarioPorHora = (s >= 0) ? s : 0;
17 }
18 double FuncionarioHorista::getSalarioPorHora() const { return salarioPorHora; }
19
20 void FuncionarioHorista::setHoras(double h) {
21     horasSemanais = (h >= 0 && h <= 168) ? h : 0;
22 }
23 double FuncionarioHorista::getHoras() const { return horasSemanais; }
24
25 // Padrao do livro: horas alem de 40 (por semana) recebem 1.5x.
26 // Multiplicamos por 4 para o mes.
27 double FuncionarioHorista::earnings() const {
28     double semanal;
29     if (horasSemanais <= 40)
30         semanal = salarioPorHora * horasSemanais;
31     else
32         semanal = 40 * salarioPorHora
33                 + (horasSemanais - 40) * salarioPorHora * 1.5;
34     return semanal * 4;
35 }
36

```

```

37 void FuncionarioHorista::print() const {
38     cout << "Horista: ";
39     Funcionario::print();
40     cout << "\n Salario/hora: R$ "
41         << fixed << setprecision(2) << salarioPorHora
42         << ", horas semanais: " << horasSemanais;
43 }

```

```

1  // Exercício 21.12 - FuncionarioComissao.h
2  #ifndef FUNCCOMISSAO_H
3  #define FUNCCOMISSAO_H
4
5  #include "Funcionario.h"
6
7  class FuncionarioComissao : public Funcionario {
8  public:
9      FuncionarioComissao(const std::string &p, const std::string &s,
10                          const std::string &c, const Date &nasc,
11                          double vendasBrutas, double taxaComissao);
12
13      void setVendasBrutas(double v);    // >= 0
14      double getVendasBrutas() const;
15      void setTaxaComissao(double t);    // 0 < t < 1
16      double getTaxaComissao() const;
17
18      double earnings() const override;
19      void print() const override;
20
21  private:
22      double vendasBrutas;    // mensal
23      double taxaComissao;    // 0 a 1
24 };
25
26 #endif

```

```

1  // Exercício 21.12 - FuncionarioComissao.cpp
2  #include <iostream>
3  #include <iomanip>
4  #include "FuncionarioComissao.h"
5  using namespace std;
6
7  FuncionarioComissao::FuncionarioComissao(
8      const string &p, const string &s, const string &c,
9      const Date &nasc, double v, double t)
10      : Funcionario(p, s, c, nasc) {
11      setVendasBrutas(v);
12      setTaxaComissao(t);
13  }
14
15  void FuncionarioComissao::setVendasBrutas(double v) {
16      vendasBrutas = (v >= 0) ? v : 0;
17  }
18  double FuncionarioComissao::getVendasBrutas() const { return vendasBrutas; }
19
20  void FuncionarioComissao::setTaxaComissao(double t) {
21      taxaComissao = (t > 0 && t < 1) ? t : 0;
22  }
23  double FuncionarioComissao::getTaxaComissao() const { return taxaComissao; }
24
25  double FuncionarioComissao::earnings() const {
26      return vendasBrutas * taxaComissao;
27  }
28

```



```

29 void FuncionarioComissao::print() const {
30     cout << "Comissionado: ";
31     Funcionario::print();
32     cout << "\n Vendas: R$ "
33         << fixed << setprecision(2) << vendasBrutas
34         << ", taxa: " << taxaComissao;
35 }

```

```

1  // Exercício 21.12 - FuncionarioBaseMaisComissao.h
2  // Herda de FuncionarioComissao e adiciona salario base
3  #ifndef FUNCBASEMAISCOMISSAO_H
4  #define FUNCBASEMAISCOMISSAO_H
5
6  #include "FuncionarioComissao.h"
7
8  class FuncionarioBaseMaisComissao : public FuncionarioComissao {
9  public:
10     FuncionarioBaseMaisComissao(const std::string &p, const std::string &s,
11                                const std::string &c, const Date &nasc,
12                                double vendasBrutas, double taxaComissao,
13                                double salarioBase);
14
15     void setSalarioBase(double sb);
16     double getSalarioBase() const;
17
18     // Redefine: soma salario base aa comissao
19     double earnings() const override;
20     void print() const override;
21
22 private:
23     double salarioBase;
24 };
25
26 #endif

```

```

1  // Exercício 21.12 - FuncionarioBaseMaisComissao.cpp
2  #include <iostream>
3  #include <iomanip>
4  #include "FuncionarioBaseMaisComissao.h"
5  using namespace std;
6
7  FuncionarioBaseMaisComissao::FuncionarioBaseMaisComissao(
8      const string &p, const string &s, const string &c,
9      const Date &nasc, double v, double t, double sb)
10      : FuncionarioComissao(p, s, c, nasc, v, t) {
11      setSalarioBase(sb);
12  }
13
14  void FuncionarioBaseMaisComissao::setSalarioBase(double sb) {
15      salarioBase = (sb >= 0) ? sb : 0;
16  }
17  double FuncionarioBaseMaisComissao::getSalarioBase() const {
18      return salarioBase;
19  }
20
21  // Chama o earnings() da base via :: para reusar a logica de comissao
22  double FuncionarioBaseMaisComissao::earnings() const {
23      return FuncionarioComissao::earnings() + salarioBase;
24  }
25
26  void FuncionarioBaseMaisComissao::print() const {
27      cout << "Base+Comissao: ";
28      Funcionario::print();

```

```

29     cout << "\n  Vendas: R$ "
30         << fixed << setprecision(2) << getVendasBrutas()
31         << ", taxa: " << getTaxaComissao()
32         << "\n  Salario base: R$ " << salarioBase;
33 }

```

Programa principal — folha polimórfica com bônus

```

1  // Exercício 21.12 - main_Funcionario.cpp
2  // Folha de pagamento polimorfica com bonus de aniversario
3  #include <iostream>
4  #include <iomanip>
5  #include <vector>
6  #include <ctime>
7  #include "Funcionario.h"
8  #include "FuncionarioAssalariado.h"
9  #include "FuncionarioHorista.h"
10 #include "FuncionarioComissao.h"
11 #include "FuncionarioBaseMaisComissao.h"
12 using namespace std;
13
14 int main() {
15     cout << fixed << setprecision(2);
16
17     // Cria os funcionarios
18     FuncionarioAssalariado f1("Joao", "Silva", "111.111.111-11",
19                               Date(5, 15, 1985), 800.00);
20     FuncionarioHorista      f2("Maria", "Santos", "222.222.222-22",
21                               Date(11, 20, 1990), 25.00, 45);
22     FuncionarioComissao     f3("Pedro", "Lima", "333.333.333-33",
23                               Date(7, 8, 1980), 10000.00, 0.06);
24     FuncionarioBaseMaisComissao f4("Ana", "Costa", "444.444.444-44",
25                                   Date(3, 22, 1992), 8000.00, 0.04, 500.00);
26
27     // Vector de REFERENCIAS a Funcionario (como pede o enunciado)
28     // Usamos reference_wrapper porque vector nao aceita refs nativas.
29     vector<reference_wrapper<Funcionario>> folha = { f1, f2, f3, f4 };
30
31     // Descobre o mes atual do sistema
32     time_t agora = time(nullptr);
33     struct tm *partes = localtime(&agora);
34     int mesAtual = partes->tm_mon + 1;
35     cout << "=== Folha de pagamento (mes atual: " << mesAtual << ") ==\n\n";
36
37     double totalFolha = 0;
38     for (Funcionario &f : folha) {
39         f.print();
40         double pagamento = f.earnings(); // POLIMORFISMO!
41         cout << "\n  Pagamento bruto: R$ " << pagamento;
42
43         // Bonus de aniversario: +R$ 100 se mes atual = mes de nascimento
44         if (f.getDataNascimento().getMes() == mesAtual) {
45             cout << "\n  *** BONUS DE ANIVERSARIO: +R$ 100.00 ***";
46             pagamento += 100.00;
47         }
48         cout << "\n  Pagamento final: R$ " << pagamento << "\n\n";
49         totalFolha += pagamento;
50     }
51
52     cout << "Total da folha de pagamento: R$ " << totalFolha << endl;
53
54     return 0;

```

55 }

Execução

Saída da execução

```

=== Folha de pagamento (mes atual: 5) ===

Assalariado: Joao Silva
  CPF: 111.111.111-11
  Nascimento: 15/5/1985
  Salario semanal: R$ 800.00
  Pagamento bruto: R$ 3200.00
  *** BONUS DE ANIVERSARIO: +R$ 100.00 ***
  Pagamento final: R$ 3300.00

Horista: Maria Santos
  CPF: 222.222.222-22
  Nascimento: 20/11/1990
  Salario/hora: R$ 25.00, horas semanais: 45.00
  Pagamento bruto: R$ 4750.00
  Pagamento final: R$ 4750.00

Comissionado: Pedro Lima
  CPF: 333.333.333-33
  Nascimento: 8/7/1980
  Vendas: R$ 10000.00, taxa: 0.06
  Pagamento bruto: R$ 600.00
  Pagamento final: R$ 600.00

Base+Comissao: Ana Costa
  CPF: 444.444.444-44
  Nascimento: 22/3/1992
  Vendas: R$ 8000.00, taxa: 0.04
  Salario base: R$ 500.00
  Pagamento bruto: R$ 820.00
  Pagamento final: R$ 820.00

Total da folha de pagamento: R$ 9470.00

```

Conferência matemática

- Joao (Assalariado, R\$ 800/sem, aniversário maio): $800 \times 4 = 3200 + 100 \text{ bônus} = 3300$.
- Maria (Horista, 45h/sem): $40 \times 25 + 5 \times 25 \times 1,5 = 1000 + 187,5 = 1187,5/\text{sem}$; mensal = 4750.
- Pedro (Comissionado, 6% de R\$ 10000): 600.
- Ana (Base+Comissão, 4% de R\$ 8000 + R\$ 500): $320 + 500 = 820$.
- **Total:** $3300 + 4750 + 600 + 820 = 9470$.

Discussão — pontos técnicos importantes

- **Vector de referências:** vector em C++ não aceita referências nativas como tipo de elemento. Usamos `std::reference_wrapper<Funcionario>`, que envolve uma referência em um objeto copiável. O `for (Funcionario &f : folha)` faz *unwrap* automático.

- **Despacho dinâmico:** a chamada `f.earnings()` chama a versão correta de cada derivada via `vtable`. Sem `virtual` na base, seria chamada `Funcionario::earnings()` — que não existe (pura)!
- **Destrutor virtual:** mesmo não destruindo os objetos via ponteiro à base aqui (são locais), inclui `virtual ~Funcionario()` por boa prática.

11 Exercício 21.13 — Hierarquia Forma com vector polimórfico

Enunciado. Implemente a hierarquia Forma. Cada FormaBidimensional deve ter `retArea()`; cada FormaTridimensional deve ter `retArea()` e `retVolume()`. Use um vetor de ponteiros Forma* para objetos concretos. Para cada forma, descobrir se é 2D ou 3D e imprimir as medidas relevantes.

Classes-base abstratas

```

1 // Exercício 21.13 - Forma.h
2 // Classe-base abstrata para a hierarquia de formas geometricas
3 #ifndef FORMA_H
4 #define FORMA_H
5
6 #include <string>
7 #include <iostream>
8
9 class Forma {
10 public:
11     Forma(const std::string &nome);
12     virtual ~Forma() {} // destrutor virtual
13
14     std::string getNome() const;
15
16     // Funcao virtual pura: cada derivada CONCRETA define
17     virtual void print() const = 0;
18
19 private:
20     std::string nome;
21 };
22
23 #endif

```

```

1 // Exercício 21.13 - Forma.cpp
2 #include "Forma.h"
3 using namespace std;
4
5 Forma::Forma(const string &n) : nome(n) {}
6
7 string Forma::getNome() const { return nome; }

```

```

1 // Exercício 21.13 - FormaBidimensional.h
2 // Camada intermediaria ABSTRATA: adiciona retArea()
3 #ifndef FORMA2D_H
4 #define FORMA2D_H
5
6 #include "Forma.h"
7
8 class FormaBidimensional : public Forma {
9 public:
10     FormaBidimensional(const std::string &nome);
11
12     virtual double retArea() const = 0; // virtual pura: ainda abstrata
13 };
14
15 #endif

```

```

1 // Exercício 21.13 - FormaTridimensional.h
2 // Camada intermediaria ABSTRATA: adiciona retArea() e retVolume()

```

```

3  #ifndef FORMA3D_H
4  #define FORMA3D_H
5
6  #include "Forma.h"
7
8  class FormaTridimensional : public Forma {
9  public:
10     FormaTridimensional(const std::string &nome);
11
12     virtual double retArea() const = 0;
13     virtual double retVolume() const = 0;
14 };
15
16 #endif

```

Formas concretas bidimensionais

```

1  // Exercício 21.13 - Circulo.h
2  #ifndef CIRCULO_H
3  #define CIRCULO_H
4
5  #include "FormaBidimensional.h"
6
7  class Circulo : public FormaBidimensional {
8  public:
9     Circulo(double raio);
10
11     double retArea() const override;
12     void print() const override;
13
14 private:
15     double raio;
16 };
17
18 #endif

```

```

1  // Exercício 21.13 - Circulo.cpp
2  #include <iostream>
3  #include <iomanip>
4  #include <cmath>
5  #include "Circulo.h"
6  using namespace std;
7
8  Circulo::Circulo(double r)
9     : FormaBidimensional("Circulo"), raio(r >= 0 ? r : 0) {}
10
11 double Circulo::retArea() const {
12     return M_PI * raio * raio;
13 }
14
15 void Circulo::print() const {
16     cout << "Circulo de raio " << fixed << setprecision(2) << raio;
17 }

```

```

1  // Exercício 21.13 - Quadrado.h
2  #ifndef QUADRADO_H
3  #define QUADRADO_H
4
5  #include "FormaBidimensional.h"
6
7  class Quadrado : public FormaBidimensional {

```

```

8 public:
9     Quadrado(double lado);
10
11     double retArea() const override;
12     void print() const override;
13
14 private:
15     double lado;
16 };
17
18 #endif

```

```

1 // Exercício 21.13 - Quadrado.cpp
2 #include <iostream>
3 #include <iomanip>
4 #include "Quadrado.h"
5 using namespace std;
6
7 Quadrado::Quadrado(double l)
8     : FormaBidimensional("Quadrado"), lado(l >= 0 ? l : 0) {}
9
10 double Quadrado::retArea() const { return lado * lado; }
11
12 void Quadrado::print() const {
13     cout << "Quadrado de lado " << fixed << setprecision(2) << lado;
14 }

```

```

1 // Exercício 21.13 - Triangulo.h
2 #ifndef TRIANGULO_H
3 #define TRIANGULO_H
4
5 #include "FormaBidimensional.h"
6
7 class Triangulo : public FormaBidimensional {
8 public:
9     Triangulo(double base, double altura);
10
11     double retArea() const override;
12     void print() const override;
13
14 private:
15     double base;
16     double altura;
17 };
18
19 #endif

```

```

1 // Exercício 21.13 - Triangulo.cpp
2 #include <iostream>
3 #include <iomanip>
4 #include "Triangulo.h"
5 using namespace std;
6
7 Triangulo::Triangulo(double b, double h)
8     : FormaBidimensional("Triangulo"),
9     base(b >= 0 ? b : 0), altura(h >= 0 ? h : 0) {}
10
11 double Triangulo::retArea() const { return 0.5 * base * altura; }
12
13 void Triangulo::print() const {
14     cout << "Triangulo base=" << fixed << setprecision(2) << base
15         << " altura=" << altura;
16 }

```

Formas concretas tridimensionais

```

1 // Exercício 21.13 - Esfera.h
2 #ifndef ESFERA_H
3 #define ESFERA_H
4
5 #include "FormaTridimensional.h"
6
7 class Esfera : public FormaTridimensional {
8 public:
9     Esfera(double raio);
10
11     double retArea()    const override;
12     double retVolume()  const override;
13     void  print()       const override;
14
15 private:
16     double raio;
17 };
18
19 #endif

```

```

1 // Exercício 21.13 - Esfera.cpp
2 #include <iostream>
3 #include <iomanip>
4 #include <cmath>
5 #include "Esfera.h"
6 using namespace std;
7
8 Esfera::Esfera(double r)
9     : FormaTridimensional("Esfera"), raio(r >= 0 ? r : 0) {}
10
11 // Area da superficie: 4 * pi * r^2
12 double Esfera::retArea() const {
13     return 4 * M_PI * raio * raio;
14 }
15 // Volume: (4/3) * pi * r^3
16 double Esfera::retVolume() const {
17     return (4.0 / 3.0) * M_PI * raio * raio * raio;
18 }
19
20 void Esfera::print() const {
21     cout << "Esfera de raio " << fixed << setprecision(2) << raio;
22 }

```

```

1 // Exercício 21.13 - Cubo.h
2 #ifndef CUBO_H
3 #define CUBO_H
4
5 #include "FormaTridimensional.h"
6
7 class Cubo : public FormaTridimensional {
8 public:
9     Cubo(double lado);
10
11     double retArea()    const override;
12     double retVolume()  const override;
13     void  print()       const override;
14
15 private:
16     double lado;
17 };
18

```



```
19 #endif
```

```
1 // Exercício 21.13 - Cubo.cpp
2 #include <iostream>
3 #include <iomanip>
4 #include "Cubo.h"
5 using namespace std;
6
7 Cubo::Cubo(double l)
8     : FormaTridimensional("Cubo"), lado(l >= 0 ? l : 0) {}
9
10 // 6 faces quadradas
11 double Cubo::retArea() const { return 6 * lado * lado; }
12 double Cubo::retVolume() const { return lado * lado * lado; }
13
14 void Cubo::print() const {
15     cout << "Cubo de lado " << fixed << setprecision(2) << lado;
16 }
```

```
1 // Exercício 21.13 - Cilindro.h
2 #ifndef CILINDRO_H
3 #define CILINDRO_H
4
5 #include "FormaTridimensional.h"
6
7 class Cilindro : public FormaTridimensional {
8 public:
9     Cilindro(double raio, double altura);
10
11     double retArea() const override;
12     double retVolume() const override;
13     void print() const override;
14
15 private:
16     double raio;
17     double altura;
18 };
19
20 #endif
```

```
1 // Exercício 21.13 - Cilindro.cpp
2 #include <iostream>
3 #include <iomanip>
4 #include <cmath>
5 #include "Cilindro.h"
6 using namespace std;
7
8 Cilindro::Cilindro(double r, double h)
9     : FormaTridimensional("Cilindro"),
10     raio(r >= 0 ? r : 0), altura(h >= 0 ? h : 0) {}
11
12 // Area total: 2 * pi * r^2 (duas bases) + 2 * pi * r * h (lateral)
13 double Cilindro::retArea() const {
14     return 2 * M_PI * raio * (raio + altura);
15 }
16 // Volume: pi * r^2 * h
17 double Cilindro::retVolume() const {
18     return M_PI * raio * raio * altura;
19 }
20
21 void Cilindro::print() const {
22     cout << "Cilindro raio=" << fixed << setprecision(2) << raio
23         << " altura=" << altura;
```

```
24 }
```

Programa principal — vetor polimórfico com dynamic_cast

```
1  // Exercício 21.13 - main_Forma.cpp
2  #include <iostream>
3  #include <iomanip>
4  #include <vector>
5  #include <memory>
6  #include "Forma.h"
7  #include "FormaBidimensional.h"
8  #include "FormaTridimensional.h"
9  #include "Circulo.h"
10 #include "Quadrado.h"
11 #include "Triangulo.h"
12 #include "Esfera.h"
13 #include "Cubo.h"
14 #include "Cilindro.h"
15 using namespace std;
16
17 int main() {
18     cout << fixed << setprecision(4);
19
20     // Vector de ponteiros Forma para formas concretas
21     vector<unique_ptr<Forma>> formas;
22     formas.push_back(unique_ptr<Forma>(new Circulo(5.0)));
23     formas.push_back(unique_ptr<Forma>(new Quadrado(4.0)));
24     formas.push_back(unique_ptr<Forma>(new Triangulo(6.0, 4.0)));
25     formas.push_back(unique_ptr<Forma>(new Esfera(3.0)));
26     formas.push_back(unique_ptr<Forma>(new Cubo(2.0)));
27     formas.push_back(unique_ptr<Forma>(new Cilindro(2.0, 5.0)));
28
29     cout << "=== Processamento polimorfico de formas ===" << endl;
30     for (const auto &fp : formas) {
31         cout << "\n* ";
32         fp->print();    // POLIMORFISMO!
33
34         // dynamic_cast para descobrir o tipo concreto
35         FormaBidimensional *p2D = dynamic_cast<FormaBidimensional*>(fp.get());
36         FormaTridimensional *p3D = dynamic_cast<FormaTridimensional*>(fp.get());
37
38         if (p2D) {
39             cout << "\n [2D] Area = " << p2D->retArea();
40         } else if (p3D) {
41             cout << "\n [3D] Area total da superficie = " << p3D->retArea()
42                 << "\n          Volume = " << p3D->retVolume();
43         }
44     }
45     cout << endl;
46
47     return 0;
48 }
```

Execução

Saída da execução

```

=== Processamento polimorfico de formas ===

* Circulo de raio 5.00
  [2D] Area = 78.54
* Quadrado de lado 4.00
  [2D] Area = 16.00
* Triangulo base=6.00 altura=4.00
  [2D] Area = 12.00
* Esfera de raio 3.00
  [3D] Area total da superficie = 113.10
      Volume = 113.10
* Cubo de lado 2.00
  [3D] Area total da superficie = 24.00
      Volume = 8.00
* Cilindro raio=2.00 altura=5.00
  [3D] Area total da superficie = 87.96
      Volume = 62.83

```

Conferência matemática

- Círculo $r = 5$: $\pi r^2 \approx 78,54$.
- Quadrado $l = 4$: $l^2 = 16$.
- Triângulo $b = 6, h = 4$: $\frac{1}{2}bh = 12$.
- Esfera $r = 3$: superfície = $4\pi r^2 \approx 113,10$; volume = $\frac{4}{3}\pi r^3 \approx 113,10$. (curiosamente, ambos coincidem para $r = 3$).
- Cubo $l = 2$: superfície = $6l^2 = 24$; volume = $l^3 = 8$.
- Cilindro $r = 2, h = 5$: superfície = $2\pi r(r + h) = 28\pi \approx 87,96$; volume = $\pi r^2 h = 20\pi \approx 62,83$.

Discussão

- **Uso de `dynamic_cast`:** cada `Forma*` é testado contra `FormaBidimensional*` e `FormaTridimensional*`. Em sucesso, o ponteiro convertido é não-nulo. É o jeito C++ de implementar “*instanceof*” do Java.
- **`unique_ptr`:** usamos `vector<unique_ptr<Forma>` em vez de `vector<Forma*>`. Garante destruição automática (RAII), evitando vazamentos de memória.
- **Destrutor virtual essencial:** sem `virtual ~Forma()`, deletar via ponteiro à base chamaria apenas o destrutor da base, vazando os recursos específicos da derivada.

12 Exercício 21.14 — Gerenciador de tela polimórfico

Enunciado. Desenvolva um pacote gráfico básico usando a hierarquia Forma. Limite-se a formas bidimensionais. Cada classe Forma tem sua própria `desenhar`. Escreva um gerenciador de tela polimórfico que percorra um array `Forma*`, enviando `desenhar` a cada objeto. Redesenhe a tela a cada nova forma adicionada.

Análise do problema

Esta é uma versão especializada e focada da hierarquia do 21.13. Aqui as formas são concebidas para serem *desenhadas em uma tela ASCII* de tamanho fixo (20×50). Cada forma tem posição, tamanho e caractere de preenchimento.

Classe-base abstrata

```

1 // Exercício 21.14 - FormaDesenhavel.h
2 // Classe-base abstrata para formas que podem ser desenhadas em ASCII.
3 // Cada forma tem posicao (x,y), tamanho e caractere de preenchimento.
4 #ifndef FORMADESENHABEL_H
5 #define FORMADESENHABEL_H
6
7 #include <vector>
8 #include <string>
9
10 // Representamos a tela como uma matriz de caracteres
11 using Tela = std::vector<std::vector<char>>>;
12
13 class FormaDesenhavel {
14 public:
15     FormaDesenhavel(int x, int y, char preenche);
16     virtual ~FormaDesenhavel() {}
17
18     // Funcao virtual pura: cada forma desenha-se na tela
19     virtual void desenhar(Tela &tela) const = 0;
20     virtual std::string descricao() const = 0;
21
22     int getX() const;
23     int getY() const;
24     char getPreenchimento() const;
25
26 protected:
27     int x, y;           // canto de referencia
28     char preenche;      // caractere de preenchimento
29 };
30
31 #endif

```

```

1 // Exercício 21.14 - FormaDesenhavel.cpp
2 #include "FormaDesenhavel.h"
3
4 FormaDesenhavel::FormaDesenhavel(int xi, int yi, char p)
5     : x(xi), y(yi), preenche(p) {}
6
7 int FormaDesenhavel::getX() const { return x; }
8 int FormaDesenhavel::getY() const { return y; }
9 char FormaDesenhavel::getPreenchimento() const { return preenche; }

```

Quatro formas desenháveis

```

1 // Exercício 21.14 - QuadradoDes.h
2 #ifndef QUADRADODES_H
3 #define QUADRADODES_H
4
5 #include "FormaDesenhavel.h"
6
7 class QuadradoDes : public FormaDesenhavel {
8 public:
9     QuadradoDes(int x, int y, int lado, char p);
10
11     void desenhar(Tela &tela) const override;
12     std::string descricao() const override;
13
14 private:
15     int lado;
16 };
17
18 #endif

```

```

1 // Exercício 21.14 - QuadradoDes.cpp
2 #include <sstream>
3 #include "QuadradoDes.h"
4 using namespace std;
5
6 QuadradoDes::QuadradoDes(int xi, int yi, int l, char p)
7     : FormaDesenhavel(xi, yi, p), lado(l > 0 ? l : 1) {}
8
9 void QuadradoDes::desenhar(Tela &t) const {
10     int alt = (int)t.size();
11     int lar = alt > 0 ? (int)t[0].size() : 0;
12     for (int i = 0; i < lado; ++i) {
13         for (int j = 0; j < lado; ++j) {
14             int ty = y + i;
15             int tx = x + j;
16             if (ty >= 0 && ty < alt && tx >= 0 && tx < lar)
17                 t[ty][tx] = preenche;
18         }
19     }
20 }
21
22 string QuadradoDes::descricao() const {
23     ostringstream oss;
24     oss << "Quadrado(" << x << "," << y << ") lado=" << lado
25         << " ' " << preenche << "' ";
26     return oss.str();
27 }

```

```

1 // Exercício 21.14 - RetanguloDes.h
2 #ifndef RETANGULODES_H
3 #define RETANGULODES_H
4
5 #include "FormaDesenhavel.h"
6
7 class RetanguloDes : public FormaDesenhavel {
8 public:
9     RetanguloDes(int x, int y, int larg, int alt, char p);
10
11     void desenhar(Tela &tela) const override;
12     std::string descricao() const override;
13
14 private:

```

```

15     int largura, altura;
16 };
17
18 #endif

```

```

1  // Exercício 21.14 - RetanguloDes.cpp
2  #include <sstream>
3  #include "RetanguloDes.h"
4  using namespace std;
5
6  RetanguloDes::RetanguloDes(int xi, int yi, int l, int a, char p)
7      : FormaDesenhavel(xi, yi, p),
8        largura(l > 0 ? l : 1), altura(a > 0 ? a : 1) {}
9
10 void RetanguloDes::desenhar(Tela &t) const {
11     int alt = (int)t.size();
12     int lar = alt > 0 ? (int)t[0].size() : 0;
13     for (int i = 0; i < altura; ++i) {
14         for (int j = 0; j < largura; ++j) {
15             int ty = y + i, tx = x + j;
16             if (ty >= 0 && ty < alt && tx >= 0 && tx < lar)
17                 t[ty][tx] = preenche;
18         }
19     }
20 }
21
22 string RetanguloDes::descricao() const {
23     ostringstream oss;
24     oss << "Retangulo(" << x << "," << y << ") "
25         << largura << "x" << altura << " " << preenche << "'";
26     return oss.str();
27 }

```

```

1  // Exercício 21.14 - TrianguloDes.h
2  // Triangulo retangulo com cateto na origem
3  #ifndef TRIANGULODES_H
4  #define TRIANGULODES_H
5
6  #include "FormaDesenhavel.h"
7
8  class TrianguloDes : public FormaDesenhavel {
9  public:
10     TrianguloDes(int x, int y, int altura, char p);
11
12     void desenhar(Tela &tela) const override;
13     std::string descricao() const override;
14
15 private:
16     int altura;           // tambem = base
17 };
18
19 #endif

```

```

1  // Exercício 21.14 - TrianguloDes.cpp
2  #include <sstream>
3  #include "TrianguloDes.h"
4  using namespace std;
5
6  TrianguloDes::TrianguloDes(int xi, int yi, int a, char p)
7      : FormaDesenhavel(xi, yi, p), altura(a > 0 ? a : 1) {}
8
9  // Triangulo retangulo: a primeira linha tem 1 ponto, a segunda 2, etc.
10 void TrianguloDes::desenhar(Tela &t) const {

```

```

11     int alt = (int)t.size();
12     int lar = alt > 0 ? (int)t[0].size() : 0;
13     for (int i = 0; i < altura; ++i) {
14         for (int j = 0; j <= i; ++j) {
15             int ty = y + i, tx = x + j;
16             if (ty >= 0 && ty < alt && tx >= 0 && tx < lar)
17                 t[ty][tx] = preenche;
18         }
19     }
20 }
21
22 string TrianguloDes::descricao() const {
23     ostringstream oss;
24     oss << "Triangulo(" << x << "," << y << ") altura=" << altura
25         << " ' " << preenche << "'";
26     return oss.str();
27 }

```

```

1 // Exercício 21.14 - CirculoDes.h
2 #ifndef CIRCULODES_H
3 #define CIRCULODES_H
4
5 #include "FormaDesenhavel.h"
6
7 class CirculoDes : public FormaDesenhavel {
8 public:
9     // (x, y) e o CENTRO; raio em celulas
10    CirculoDes(int x, int y, int raio, char p);
11
12    void desenhar(Tela &tela) const override;
13    std::string descricao() const override;
14
15 private:
16    int raio;
17 };
18
19 #endif

```

```

1 // Exercício 21.14 - CirculoDes.cpp
2 #include <sstream>
3 #include "CirculoDes.h"
4 using namespace std;
5
6 CirculoDes::CirculoDes(int xi, int yi, int r, char p)
7     : FormaDesenhavel(xi, yi, p), raio(r > 0 ? r : 1) {}
8
9 // Desenha aproximadamente um disco: todas as celulas (i,j) tais
10 // que (i-cx)^2 + (j-cy)^2 <= r^2 sao marcadas.
11 void CirculoDes::desenhar(Tela &t) const {
12     int alt = (int)t.size();
13     int lar = alt > 0 ? (int)t[0].size() : 0;
14     for (int i = -raio; i <= raio; ++i) {
15         for (int j = -raio; j <= raio; ++j) {
16             if (i*i + j*j <= raio*raio) {
17                 int ty = y + i, tx = x + j;
18                 if (ty >= 0 && ty < alt && tx >= 0 && tx < lar)
19                     t[ty][tx] = preenche;
20             }
21         }
22     }
23 }
24
25 string CirculoDes::descricao() const {

```

```

26     ostreamstream oss;
27     oss << "Circulo centro=(" << x << "," << y << ") raio=" << raio
28         << " ' ' " << preenche << " ' ' ";
29     return oss.str();
30 }

```

Programa principal — gerenciador de tela

```

1  // Exercício 21.14 - main_Tela.cpp
2  // Gerenciador de tela polimorfo
3  #include <iostream>
4  #include <vector>
5  #include <memory>
6  #include "FormaDesenhavel.h"
7  #include "QuadradoDes.h"
8  #include "RetanguloDes.h"
9  #include "TrianguloDes.h"
10 #include "CirculoDes.h"
11 using namespace std;
12
13 const int ALT_TELA = 20;
14 const int LAR_TELA = 50;
15
16 void limparTela(Tela &t) {
17     for (auto &linha : t)
18         for (auto &c : linha)
19             c = ' ';
20 }
21
22 void imprimirTela(const Tela &t) {
23     // Moldura superior
24     cout << "+";
25     for (int j = 0; j < (int)t[0].size(); ++j) cout << "-";
26     cout << "+\n";
27     // Conteúdo
28     for (const auto &linha : t) {
29         cout << "|";
30         for (char c : linha) cout << c;
31         cout << "|\n";
32     }
33     // Moldura inferior
34     cout << "+";
35     for (int j = 0; j < (int)t[0].size(); ++j) cout << "-";
36     cout << "+\n";
37 }
38
39 // Gerenciador: redesenha TODAS as formas a cada chamada
40 void redesenharTela(Tela &t,
41                     const vector<unique_ptr<FormaDesenhavel>> &formas) {
42     limparTela(t);
43     for (const auto &f : formas) {
44         f->desenhar(t); // CHAMADA POLIMORFICA!
45     }
46     imprimirTela(t);
47 }
48
49 int main() {
50     Tela tela(ALT_TELA, vector<char>(LAR_TELA, ' '));
51     vector<unique_ptr<FormaDesenhavel>> formas;
52
53     cout << "=== Cenário pre-roteirizado: 4 formas adicionadas ===\n";
54 }

```



```

55 formas.push_back(unique_ptr<FormaDesenhavel>(
56     new QuadradoDes(2, 2, 5, '#')));
57 cout << "\nApos adicionar " << formas.back()->descricao() << ":\n";
58 redesenharTela(tela, formas);
59
60 formas.push_back(unique_ptr<FormaDesenhavel>(
61     new RetanguloDes(15, 3, 12, 6, '*')));
62 cout << "\nApos adicionar " << formas.back()->descricao() << ":\n";
63 redesenharTela(tela, formas);
64
65 formas.push_back(unique_ptr<FormaDesenhavel>(
66     new TrianguloDes(35, 4, 8, '+')));
67 cout << "\nApos adicionar " << formas.back()->descricao() << ":\n";
68 redesenharTela(tela, formas);
69
70 formas.push_back(unique_ptr<FormaDesenhavel>(
71     new CirculoDes(15, 14, 4, '@')));
72 cout << "\nApos adicionar " << formas.back()->descricao() << ":\n";
73 redesenharTela(tela, formas);
74
75 return 0;
76 }

```

Execução (saída final, após adicionar as 4 formas)

Saída da execução

```
Apos adicionar Circulo cenro=(15,14)  raio=4  '@':
+-----+
| .....|
| .....|
| .####. ....|
| .####. ....*****|
| .####. ....*****+.|
| .####. ....*****++|
| .####. ....*****+++|
| .....*****++++|
| .....*****+++++|
| .....@.....+++++++|
| .....@@@@.....+++++++|
| .....@@@@@@|
| .....@@@@@@@|
| .....@@@@@@@@|
| .....@@@@@@@@@@|
| .....@@@@@@@@@|
| .....@@@@@@@@@|
| .....@@@@@@|
| .....@.....|
| .....|
+-----+
```

Discussão

- **O gerenciador é o “cliente polimórfico”:** a função `redesenharTela` simplesmente itera sobre o vetor, chamando `f->desenhar(tela)` em cada um. Não precisa saber *qual* forma está desenhando.
- **Redesenho completo:** cada chamada a `redesenharTela` limpa a tela e redesenha todas as formas. É o padrão usado por engines reais — redesenhar é mais simples que rastrear mudanças

incrementais (e em *frame rates* altos, não há diferença percebida).

- **Extensão trivial:** para adicionar elipses, polígonos regulares ou estrelas, basta criar a classe respectiva derivada de `FormaDesenhavel` e implementar `desenhar`. O gerenciador *não muda*.

13 Exercício 21.15 — Pacote polimórfico

Enunciado. Use a hierarquia Pacote do Ex. 20.9 para criar um programa que processe um vector de Pacote* polimorficamente. Para cada Pacote, imprima a etiqueta de endereço e chame calculaCusto. Acumule o custo total.

Análise do problema

A hierarquia Pacote do Cap. 20 já estava pronta, mas calculaCusto não era virtual — a redefinição funcionava apenas estaticamente (com o tipo conhecido em tempo de compilação). Agora corrigimos isso.

Classe-base Pacote com virtual

```

1  // Exercício 21.15 - Pacote.h
2  // Igual ao do Cap. 20, mas agora com virtual em calculaCusto()
3  // e novos getters para enderecos do remetente e destinatario
4  #ifndef PACOTE_H
5  #define PACOTE_H
6
7  #include <string>
8
9  class Pacote {
10 public:
11     Pacote(const std::string &nomeRem, const std::string &endRem,
12           const std::string &cidadeRem, const std::string &estadoRem,
13           const std::string &cepRem,
14           const std::string &nomeDest, const std::string &endDest,
15           const std::string &cidadeDest, const std::string &estadoDest,
16           const std::string &cepDest,
17           double pesoG, double custoPorG);
18
19     virtual ~Pacote() {}           // destrutor virtual
20
21     void setPeso(double p);
22     double getPeso() const;
23     void setCustoPorGrama(double c);
24     double getCustoPorGrama() const;
25
26     // === FUNCAO VIRTUAL ===
27     virtual double calculaCusto() const;
28
29     // Getters para a etiqueta polimorfica do 21.15
30     std::string getNomeRemetente() const;
31     std::string getEnderecoRemetente() const;
32     std::string getCidadeRemetente() const;
33     std::string getEstadoRemetente() const;
34     std::string getCepRemetente() const;
35     std::string getNomeDestinatario() const;
36     std::string getEnderecoDestinatario() const;
37     std::string getCidadeDestinatario() const;
38     std::string getEstadoDestinatario() const;
39     std::string getCepDestinatario() const;
40
41 protected:
42     double peso;
43     double custoPorGrama;
44
45 private:
46     std::string nomeRemetente,    enderecoRemetente;

```

```

47     std::string cidadeRemetente, estadoRemetente, cepRemetente;
48     std::string nomeDestinatario, enderecoDestinatario;
49     std::string cidadeDestinatario, estadoDestinatario, cepDestinatario;
50 };
51
52 #endif

```



```

1  // Exercício 21.15 - Pacote.cpp
2  #include "Pacote.h"
3  using namespace std;
4
5  Pacote::Pacote(const string &nR, const string &eR,
6                const string &cR, const string &esR, const string &cepR,
7                const string &nD, const string &eD,
8                const string &cD, const string &esD, const string &cepD,
9                double p, double c)
10     : nomeRemetente(nR), enderecoRemetente(eR),
11       cidadeRemetente(cR), estadoRemetente(esR), cepRemetente(cepR),
12       nomeDestinatario(nD), enderecoDestinatario(eD),
13       cidadeDestinatario(cD), estadoDestinatario(esD), cepDestinatario(cepD) {
14     setPeso(p);
15     setCustoPorGrama(c);
16 }
17
18 void Pacote::setPeso(double p) { peso = (p >= 0) ? p : 0; }
19 double Pacote::getPeso() const { return peso; }
20 void Pacote::setCustoPorGrama(double c) { custoPorGrama = (c >= 0) ? c : 0; }
21 double Pacote::getCustoPorGrama() const { return custoPorGrama; }
22
23 double Pacote::calculaCusto() const { return peso * custoPorGrama; }
24
25 string Pacote::getNomeRemetente() const { return nomeRemetente; }
26 string Pacote::getEnderecoRemetente() const { return enderecoRemetente; }
27 string Pacote::getCidadeRemetente() const { return cidadeRemetente; }
28 string Pacote::getEstadoRemetente() const { return estadoRemetente; }
29 string Pacote::getCepRemetente() const { return cepRemetente; }
30 string Pacote::getNomeDestinatario() const { return nomeDestinatario; }
31 string Pacote::getEnderecoDestinatario() const { return enderecoDestinatario; }
32 string Pacote::getCidadeDestinatario() const { return cidadeDestinatario; }
33 string Pacote::getEstadoDestinatario() const { return estadoDestinatario; }
34 string Pacote::getCepDestinatario() const { return cepDestinatario; }

```

Derivadas com override

```

1  // Exercício 21.15 - PacoteDoisDias.h
2  #ifndef PACOTEDOISDIAS_H
3  #define PACOTEDOISDIAS_H
4
5  #include "Pacote.h"
6
7  class PacoteDoisDias : public Pacote {
8  public:
9      PacoteDoisDias(const std::string &nR, const std::string &eR,
10                    const std::string &cR, const std::string &esR,
11                    const std::string &cepR,
12                    const std::string &nD, const std::string &eD,
13                    const std::string &cD, const std::string &esD,
14                    const std::string &cepD,
15                    double pesoG, double custoPorG,
16                    double taxaFixaEntrega);
17
18     void setTaxaFixaEntrega(double t);

```

```

19     double getTaxaFixaEntrega() const;
20
21     double calculaCusto() const override;    // agora com override!
22
23 private:
24     double taxaFixaEntrega;
25 };
26
27 #endif

```

```

1  // Exercício 21.15 - PacoteDoisDias.cpp
2  #include "PacoteDoisDias.h"
3
4  PacoteDoisDias::PacoteDoisDias(
5      const std::string &nR, const std::string &eR,
6      const std::string &cR, const std::string &esR, const std::string &cepR,
7      const std::string &nD, const std::string &eD,
8      const std::string &cD, const std::string &esD, const std::string &cepD,
9      double p, double c, double tf)
10      : Pacote(nR, eR, cR, esR, cepR, nD, eD, cD, esD, cepD, p, c) {
11      setTaxaFixaEntrega(tf);
12  }
13
14  void PacoteDoisDias::setTaxaFixaEntrega(double t) {
15      taxaFixaEntrega = (t >= 0) ? t : 0;
16  }
17  double PacoteDoisDias::getTaxaFixaEntrega() const { return taxaFixaEntrega; }
18
19  double PacoteDoisDias::calculaCusto() const {
20      return Pacote::calculaCusto() + taxaFixaEntrega;
21  }

```

```

1  // Exercício 21.15 - PacoteNoturno.h
2  #ifndef PACOTENOTURNO_H
3  #define PACOTENOTURNO_H
4
5  #include "Pacote.h"
6
7  class PacoteNoturno : public Pacote {
8  public:
9      PacoteNoturno(const std::string &nR, const std::string &eR,
10                  const std::string &cR, const std::string &esR,
11                  const std::string &cepR,
12                  const std::string &nD, const std::string &eD,
13                  const std::string &cD, const std::string &esD,
14                  const std::string &cepD,
15                  double pesoG, double custoPorG,
16                  double taxaAdicionalPorG);
17
18      void setTaxaAdicional(double t);
19      double getTaxaAdicional() const;
20
21      double calculaCusto() const override;
22
23 private:
24     double taxaAdicionalPorGrama;
25 };
26
27 #endif

```

```

1  // Exercício 21.15 - PacoteNoturno.cpp
2  #include "PacoteNoturno.h"
3

```

```

4 PacoteNoturno::PacoteNoturno(
5     const std::string &nR, const std::string &eR,
6     const std::string &cR, const std::string &esR, const std::string &cepR,
7     const std::string &nD, const std::string &eD,
8     const std::string &cD, const std::string &esD, const std::string &cepD,
9     double p, double c, double ta)
10 : Pacote(nR, eR, cR, esR, cepR, nD, eD, cD, esD, cepD, p, c) {
11     setTaxaAdicional(ta);
12 }
13
14 void PacoteNoturno::setTaxaAdicional(double t) {
15     taxaAdicionalPorGrama = (t >= 0) ? t : 0;
16 }
17 double PacoteNoturno::getTaxaAdicional() const { return taxaAdicionalPorGrama; }
18
19 double PacoteNoturno::calculaCusto() const {
20     return peso * (custoPorGrama + taxaAdicionalPorGrama);
21 }

```

Programa principal

```

1 // Exercício 21.15 - main_Pacote.cpp
2 // Processa um vetor<Pacote> polimorficamente: imprime etiquetas
3 // e acumula o custo total de envio.
4 #include <iostream>
5 #include <iomanip>
6 #include <vector>
7 #include <memory>
8 #include "Pacote.h"
9 #include "PacoteDoisDias.h"
10 #include "PacoteNoturno.h"
11 using namespace std;
12
13 // Imprime uma "etiqueta" como apareceria no envelope
14 void imprimirEtiqueta(const Pacote &p) {
15     cout << "+-----+\n";
16     cout << "| DE: " << p.getNomeRemetente() << "\n";
17     cout << "| " << p.getEnderecoRemetente() << "\n";
18     cout << "| " << p.getCidadeRemetente() << "- "
19         << p.getEstadoRemetente() << " " << p.getCepRemetente() << "\n";
20     cout << "|-----|\n";
21     cout << "| PARA: " << p.getNomeDestinatario() << "\n";
22     cout << "| " << p.getEnderecoDestinatario() << "\n";
23     cout << "| " << p.getCidadeDestinatario() << "- "
24         << p.getEstadoDestinatario() << " "
25         << p.getCepDestinatario() << "\n";
26     cout << "+-----+\n";
27 }
28
29 int main() {
30     cout << fixed << setprecision(2);
31
32     vector<unique_ptr<Pacote>> remessas;
33     remessas.push_back(unique_ptr<Pacote>(new Pacote(
34         "Maria Souza", "Rua A, 100", "Sao Paulo", "SP", "01000-000",
35         "Joao Silva", "Av. B, 250", "Rio de Janeiro", "RJ", "20000-000",
36         1500.0, 0.02)));
37     remessas.push_back(unique_ptr<Pacote>(new PacoteDoisDias(
38         "Ana Costa", "Rua C, 5", "Belo Horizonte", "MG", "30000-000",
39         "Pedro Lima", "Av. D, 8", "Salvador", "BA", "40000-000",
40         2000.0, 0.025, 15.00)));
41     remessas.push_back(unique_ptr<Pacote>(new PacoteNoturno(

```

```

42     "Carlos Mendes", "Rua E, 12", "Recife", "PE", "50000-000",
43     "Lucia Pereira", "Av. F, 20", "Curitiba", "PR", "80000-000",
44     500.0, 0.03, 0.01)));
45
46     double totalEnvio = 0;
47     int i = 1;
48     for (const auto &p : remessas) {
49         cout << "\n>>> REMESSA " << i++ << " <<<\n";
50         imprimirEtiqueta(*p);
51         double custo = p->calculaCusto();    // POLIMORFISMO!
52         cout << "Custo de envio: R$ " << custo << "\n";
53         totalEnvio += custo;
54     }
55
56     cout << "\n=====\\n";
57     cout << "TOTAL DE TODAS AS REMESSAS: R$ " << totalEnvio << "\n";
58     cout << "=====\\n";
59     return 0;
60 }

```

Execução (saída parcial)

Saída da execução

```

>>> REMESSA 1 <<<
+-----+
| DE:  Maria Souza
|      Rua A, 100
|      Sao Paulo-SP  01000-000
|-----|
| PARA: Joao Silva
|      Av. B, 250
|      Rio de Janeiro-RJ  20000-000
+-----+
Custo de envio: R$ 30.00

>>> REMESSA 2 <<<
[... PacoteDoisDias: Ana Costa -> Pedro Lima ...]
Custo de envio: R$ 65.00

>>> REMESSA 3 <<<
[... PacoteNoturno: Carlos Mendes -> Lucia Pereira ...]
Custo de envio: R$ 20.00

=====
TOTAL DE TODAS AS REMESSAS: R$ 115.00
=====

```

Conferência matemática

- Pacote: $1500 \times 0,02 = 30,00$.
- PacoteDoisDias: $2000 \times 0,025 + 15,00 = 65,00$.
- PacoteNoturno: $500 \times (0,03 + 0,01) = 20,00$.
- **Total:** $30 + 65 + 20 = 115$.

Discussão

A chamada `p->calculaCusto()` ilustra o coracao do polimorfismo: três `Pacote*` apontam para tipos concretos distintos (`Pacote`, `PacoteDoisDias`, `PacoteNoturno`), e a `vtable` de cada um direciona a chamada para a versão correta. O loop *não precisa* distinguir os tipos — a hierarquia faz isso.

Observação de Engenharia de Software

Compare este código com o que escrevíamos no Cap. 20 (sem `virtual`): lá, era necessário *não* usar `Pacote*`, mas sim conhecer o tipo concreto. A inclusão de `virtual` é a mudança mínima que separa um código “adoidado pelo `switch`” de um código elegante.

14 Exercício 21.16 — Banco polimórfico

Enunciado. Use a hierarquia Conta do Ex. 20.10 para um programa bancário polimórfico. Para cada Conta, permita debitar/creditar. Se for ContaPoupanca, calcular e creditar os juros via `calculaJuros + creditar`. Imprima saldo atualizado.

Análise do problema

Igual ao 21.15: a hierarquia do Cap. 20 ganha `virtual` nos métodos sobreponíveis. Adicionalmente, usamos `dynamic_cast` para descobrir contas-poupança entre as genéricas.

Classe-base Conta com virtual

```

1 // Exercício 21.16 - Conta.h
2 // Agora com virtual em creditar() e debitar() para suportar
3 // polimorfismo dinâmico nas derivadas.
4 #ifndef CONTA_H
5 #define CONTA_H
6
7 class Conta {
8 public:
9     Conta(double saldoInicial);
10    virtual ~Conta() {}
11
12    virtual void creditar(double valor);
13    virtual bool debitar(double valor);    // true = sucesso
14    double retSaldo() const;
15
16 protected:
17     double saldo;
18 };
19
20 #endif

```

```

1 // Exercício 21.16 - Conta.cpp
2 #include <iostream>
3 #include "Conta.h"
4 using namespace std;
5
6 Conta::Conta(double saldoInicial) {
7     if (saldoInicial >= 0.0) saldo = saldoInicial;
8     else {
9         saldo = 0.0;
10        cout << "Erro: saldo inicial invalido. Saldo = 0.\n";
11    }
12 }
13
14 void Conta::creditar(double valor) { saldo += valor; }
15
16 bool Conta::debitar(double valor) {
17     if (valor > saldo) {
18         cout << "Valor do debito superior ao saldo da conta\n";
19         return false;
20     }
21     saldo -= valor;
22     return true;
23 }
24
25 double Conta::retSaldo() const { return saldo; }

```

ContaPoupanca

```

1 // Exercício 21.16 - ContaPoupanca.h
2 #ifndef CONTAPOUPANCA21_H
3 #define CONTAPOUPANCA21_H
4
5 #include "Conta.h"
6
7 class ContaPoupanca : public Conta {
8 public:
9     ContaPoupanca(double saldoInicial, double taxaJurosAnual);
10
11     void setTaxaJuros(double t);
12     double getTaxaJuros() const;
13     double calculaJuros() const;
14
15 private:
16     double taxaJurosAnual;
17 };
18
19 #endif

```

```

1 // Exercício 21.16 - ContaPoupanca.cpp
2 #include "ContaPoupanca.h"
3
4 ContaPoupanca::ContaPoupanca(double saldoInicial, double t)
5 : Conta(saldoInicial) {
6     setTaxaJuros(t);
7 }
8
9 void ContaPoupanca::setTaxaJuros(double t) {
10     taxaJurosAnual = (t >= 0.0) ? t : 0.0;
11 }
12 double ContaPoupanca::getTaxaJuros() const { return taxaJurosAnual; }
13
14 double ContaPoupanca::calculaJuros() const {
15     return saldo * taxaJurosAnual;
16 }

```

ContaCorrente com override

```

1 // Exercício 21.16 - ContaCorrente.h
2 #ifndef CONTACORRENTE21_H
3 #define CONTACORRENTE21_H
4
5 #include "Conta.h"
6
7 class ContaCorrente : public Conta {
8 public:
9     ContaCorrente(double saldoInicial, double taxaTransacao);
10
11     void setTaxaTransacao(double t);
12     double getTaxaTransacao() const;
13
14     void creditar(double valor) override;
15     bool debitar(double valor) override;
16
17 private:
18     double taxaPorTransacao;
19 };
20

```

```

21 #endif

1 // Exercício 21.16 - ContaCorrente.cpp
2 #include "ContaCorrente.h"
3
4 ContaCorrente::ContaCorrente(double saldoInicial, double t)
5 : Conta(saldoInicial) {
6     setTaxaTransacao(t);
7 }
8
9 void ContaCorrente::setTaxaTransacao(double t) {
10     taxaPorTransacao = (t >= 0.0) ? t : 0.0;
11 }
12 double ContaCorrente::getTaxaTransacao() const { return taxaPorTransacao; }
13
14 void ContaCorrente::creditar(double valor) {
15     Conta::creditar(valor);
16     saldo -= taxaPorTransacao;
17 }
18
19 bool ContaCorrente::debitar(double valor) {
20     bool ok = Conta::debitar(valor);
21     if (ok) saldo -= taxaPorTransacao;
22     return ok;
23 }

```

Programa principal

```

1 // Exercício 21.16 - main_Conta.cpp
2 // Processa um vector<Conta*> polimorficamente.
3 #include <iostream>
4 #include <iomanip>
5 #include <vector>
6 #include <memory>
7 #include "Conta.h"
8 #include "ContaPoupanca.h"
9 #include "ContaCorrente.h"
10 using namespace std;
11
12 int main() {
13     cout << fixed << setprecision(2);
14
15     vector<unique_ptr<Conta>> contas;
16     contas.push_back(unique_ptr<Conta>(new ContaPoupanca(2000.00, 0.03)));
17     contas.push_back(unique_ptr<Conta>(new ContaCorrente(1000.00, 2.00)));
18     contas.push_back(unique_ptr<Conta>(new ContaPoupanca(5000.00, 0.05)));
19
20     // Cenário pre-roteirizado: cada conta recebe um depósito e um saque
21     struct Operacao { double deposito; double saque; };
22     Operacao ops[] = { {500, 100}, {200, 300}, {1000, 500} };
23
24     cout << "=== Processamento polimorfo de contas ===\n";
25     for (size_t i = 0; i < contas.size(); ++i) {
26         Conta &c = *contas[i];
27         cout << "\n--- Conta " << (i + 1) << " ---\n";
28         cout << "Saldo inicial: R$ " << c.retSaldo() << "\n";
29
30         c.creditar(ops[i].deposito); // POLIMORFICO
31         cout << "Apos depositar R$ " << ops[i].deposito
32             << ": R$ " << c.retSaldo() << "\n";
33
34         c.debitar(ops[i].saque); // POLIMORFICO

```

```

35     cout << "Apos sacar      R$ " << ops[i].saque
36         << ": R$ " << c.retSaldo() << "\n";
37
38     // dynamic_cast para detectar ContaPoupanca e creditar juros
39     ContaPoupanca *cp = dynamic_cast<ContaPoupanca*>(&c);
40     if (cp) {
41         double juros = cp->calculaJuros();
42         cout << "Juros calculados: R$ " << juros << "\n";
43         c.creditar(juros);
44         cout << "Apos credito de juros: R$ " << c.retSaldo() << "\n";
45     }
46 }
47
48 return 0;
49 }

```

Execução

Saída da execução

```

=== Processamento polimorfico de contas ===

--- Conta 1 ---
Saldo inicial: R$ 2000.00
Apos depositar R$ 500.00: R$ 2500.00
Apos sacar      R$ 100.00: R$ 2400.00
Juros calculados: R$ 72.00
Apos credito de juros: R$ 2472.00

--- Conta 2 ---
Saldo inicial: R$ 1000.00
Apos depositar R$ 200.00: R$ 1198.00
Apos sacar      R$ 300.00: R$ 896.00

--- Conta 3 ---
Saldo inicial: R$ 5000.00
Apos depositar R$ 1000.00: R$ 6000.00
Apos sacar      R$ 500.00: R$ 5500.00
Juros calculados: R$ 275.00
Apos credito de juros: R$ 5775.00

```

Conferência matemática

- **Conta 1** (poupança 3%): $2000 + 500 - 100 = 2400$; $\text{juros} = 2400 \times 0,03 = 72$; saldo final = 2472.
- **Conta 2** (corrente, taxa R\$ 2): $1000 + 200 - 2 = 1198$; $1198 - 300 - 2 = 896$.
- **Conta 3** (poupança 5%): $5000 + 1000 - 500 = 5500$; $\text{juros} = 5500 \times 0,05 = 275$; saldo final = 5775.

Discussão

- `ContaCorrente::creditar` é a versão polimórfica chamada para `Conta&`, cobrando a taxa apropriadamente.
- `dynamic_cast<ContaPoupanca*>` retorna `nullptr` para contas correntes, não-nulo para poupanca. É o jeito C++ de fazer “se for poupança, faça isso”.

- **Conta corrente não recebe juros:** a estrutura `if (cp)` garante que apenas poupanças têm juros somados. Código cliente não precisa de `switch` explícito sobre tipo.

Encerramento

Os 14 exercícios deste capítulo cobriram todo o território do polimorfismo:

- Conceitos (21.3–21.11): por que polimorfismo, como funciona, vantagens sobre `switch`, vinculação dinâmica, *vtable*.
- Aplicações (21.12–21.16): folha de pagamento com 4 derivadas + bônus de aniversário; hierarquia de formas geométricas com despacho a `retArea/retVolume`; gerenciador de tela ASCII com formas auto-desenháveis; entrega de pacotes; banco com poupanças e correntes.

Cinco classes-base abstratas, 14 classes derivadas concretas, todas funcionando com `virtual`, `override` e `dynamic_cast`. O Capítulo 22 abordará o último grande tópico de OO em C++: **templates**, que oferecem *polimorfismo estático* (em tempo de compilação) como complemento ao dinâmico que acabamos de dominar.
